

# Trở về điểm xuất phát: một kiểu tư duy đột phá

Zhu Zeyuan

2005

*Nguồn gốc: Back to the starting point: a breakthrough way of thinking*

IOI China candidate team papers / 2005 / Zhu Zeyuan / Zhu Zeyuan.doc

## Tóm tắt

Ở mức cao, thi tin học không còn chỉ là cuộc so tài giữa một thuật toán hay một cấu trúc dữ liệu đơn lẻ, mà là phép thử đối với phương pháp tư duy và năng lực sáng tạo. Bài viết này phân tích một kiểu tư duy đột phá: **trở về điểm xuất phát**, tức là dám tạm bỏ kết quả đang có để quay lại bước phân tích ban đầu và suy nghĩ một cách tỉnh táo.

Bài viết bắt đầu từ một bài toán thông thường vốn đã có lời giải kinh điển. Sau khi nhắc lại lời giải đó, ta quay lại thuật toán thô ban đầu, tìm ra một thuật toán đơn giản đã bị bỏ quên, rồi tối ưu và liên hệ nó với rừng tập rời nhau. Cuối cùng bài viết nhấn mạnh ý nghĩa của việc tiến lên thông qua những đoạn đường vòng trong tư duy.

## 1 Đặt vấn đề

### 1.1 Mô tả bài toán

Bài toán là một biến thể của *USACO 2.1 Shaping Regions*. Có  $N$  hình chữ nhật đục màu khác nhau, với  $1 \leq N \leq 3000$ , được đặt lên một tờ giấy trắng có chiều dài  $A$  và chiều rộng  $B$ . Mọi cạnh của các hình chữ nhật đều song song với cạnh của tờ giấy, và mọi hình chữ nhật đều nằm trong tờ giấy. Gốc tọa độ  $(0, 0)$  nằm ở góc trái dưới của tờ giấy.

**Dữ liệu vào.** Dòng đầu chứa  $A, B, N$ . Từ dòng thứ hai đến dòng  $N + 1$ , các hình chữ nhật được cho theo thứ tự từ dưới lên trên. Mỗi dòng gồm năm giá trị

$$llx, lly, urx, ury, color,$$

là tọa độ góc trái dưới, tọa độ góc phải trên và màu của hình chữ nhật. Mọi tọa độ, kể cả  $A, B$ , là số thực trong đoạn từ 0 đến  $10^9$ . Màu 1 trùng với màu trắng của nền giấy.

**Dữ liệu ra.** In danh sách các màu nhìn thấy được cùng diện tích tương ứng, giữ 3 chữ số sau dấu thập phân. Danh sách được sắp theo thứ tự tăng của mã màu; không in màu có diện tích bằng 0.

Ví dụ.

```
20 20 3
2 2 18 18 2
0 8 19 19 3
8 0 10 19 4
```

```
1 91.000
2 84.000
3 187.000
4 38.000
```

## 1.2 Phân tích ban đầu: rời rạc hóa

Từ bài *Picture* của IOI 1998, các bài toán đặt hình chữ nhật trên mặt phẳng xuất hiện nhiều trong thi tin học. Tiên xử lý cơ bản nhất là rời rạc hóa mặt phẳng: lấy toàn bộ tọa độ  $x$  và  $y$  từng xuất hiện trong các hình chữ nhật, sắp xếp, loại trùng rồi đánh số theo thứ tự.

Đoạn nối hai điểm lưới kề nhau gọi là **đoạn nguyên tố**; vùng giữa hai tọa độ  $x$  kề nhau gọi là **cột rời rạc**, vùng giữa hai tọa độ  $y$  kề nhau gọi là **hàng rời rạc**; giao của một hàng và một cột rời rạc gọi là **ô rời rạc**. Mỗi tọa độ đỉnh hình chữ nhật ban đầu đều tương ứng với một tọa độ lưới.

Rời rạc hóa giúp xử lý tọa độ thực hoặc tọa độ vượt quá kiểu nguyên thông thường. Sau rời rạc hóa, mọi tọa độ đỉnh hình chữ nhật có thể biểu diễn bằng số nguyên từ 1 đến  $2N$ . Nhờ bảng tra hoặc băm, việc chuyển đổi giữa tọa độ thật và tọa độ rời rạc có thể thực hiện trong  $O(1)$ . Vì vậy phần sau chỉ xét bài toán gồm  $N$  hình chữ nhật trên lưới có tọa độ nguyên từ 1 đến  $2N$ .

## 1.3 Một ý tưởng thô

Cách thô nhất giống thuật toán loang màu. Dùng một mảng  $2N \times 2N$  ghi màu hiện tại của từng ô rời rạc, ban đầu chưa có màu. Xử lý các hình chữ nhật từ trên xuống dưới; với mỗi hình, tô màu cho phần bên dưới nó mà vẫn chưa có màu.

Nếu không đủ bộ nhớ cho mảng hai chiều, có thể xử lý từng hàng rời rạc. Với mỗi hàng, dùng mảng một chiều độ dài  $2N$ . Ta lại quét các hình chữ nhật từ trên xuống dưới; nếu hình chữ nhật cắt hàng hiện tại, tô các ô chưa màu trong đoạn giao của nó với hàng đó. Cách này hầu như không tăng thời gian, chỉ tăng một hằng số nhỏ.

# 2 Giải quyết bài toán

## 2.1 Thuật toán kinh điển

Lời giải kinh điển cho dạng bài này là cây đoạn. Với mỗi hàng rời rạc, ta dùng một cấu trúc cây  $O(N)$  bộ nhớ. Cây hỗ trợ thao tác gán thuộc tính  $(l, r, X)$ , nghĩa là gán thuộc tính  $X$  cho đoạn  $[l, r]$ .

Thao tác chuẩn trên nút  $P$  có thể mô tả như sau:

1. Nếu  $[l, r]$  không giao với đoạn của  $P$ , trả về.
2. Nếu  $[l, r]$  phủ toàn bộ đoạn của  $P$ , ghi thuộc tính  $X$  tại nút này.
3. Ngược lại, đệ quy xuống hai con của  $P$ .

Với bài toán hiện tại, thao tác cần hơi đặc biệt: chèn một đoạn màu  $X[l, r]$ , nhưng các phần đã có màu không được thay thế; sau khi xử lý xong một hàng, cần trả về tổng độ dài đang bị mỗi màu phủ.

Thao tác thống kê màu chỉ cần duyệt toàn bộ cây, mất  $O(N)$ . Thao tác chèn được sửa như sau:

1. Nếu  $[l, r]$  không giao với đoạn của  $P$ , trả về.
2. Nếu nút  $P$  đã bị một màu nào đó phủ, trả về.
3. Nếu  $[l, r]$  phủ toàn bộ đoạn của  $P$ , ghi màu  $X$  tại nút này.
4. Ngược lại, đệ quy xuống hai con của  $P$ .

Thuật toán này dùng  $O(N)$  bộ nhớ và thời gian

$$O(N^2 \log N),$$

và có thể xem là một lời giải tốt cho dữ liệu bài toán.

## 2.2 Một thuật toán khác

Bỏ qua lối mòn cây đoạn, ta quay lại điểm xuất phát và phân tích thuật toán thô ở phần trước. Một bài toán thường có hai con đường: dùng cấu trúc dữ liệu cao cấp, hoặc xem lại thuật toán ban đầu, tìm phần dư thừa rồi cải tiến đúng chỗ.

Điểm dư thừa rất rõ: khi xử lý một đoạn hiện tại, nếu gặp vùng đã được phủ, thuật toán thô không thể nhảy qua nhanh mà vẫn phải so sánh từng ô. Ta lập mảng  $next$ , ban đầu  $next[i] = i + 1$ , và mảng  $color$ , trong đó  $color[i]$  ghi màu của ô rồi rạc thứ  $i$  trên hàng hiện tại. Khi chèn đoạn  $[l, r]$ , đoạn này chiếm các ô  $l, l + 1, \dots, r - 1$ .

Thuật toán:

1. Đặt con trỏ  $i = l$ .
2. Nếu  $color[i]$  chưa có màu, tô nó bằng màu hiện tại.
3. Di chuyển  $i$  đến  $next[i]$ , đồng thời ghi lại các vị trí con trỏ đã đi qua.
4. Nếu  $i \leq r - 1$ , quay lại bước 2.
5. Sau khi kết thúc, đặt  $next[x]$  của mọi vị trí  $x$  đã đi qua thành  $next[r - 1]$ .

Ví dụ, sau khi phủ màu 1 trên  $[2, 6)$ , các ô 2, 3, 4, 5 được tô và các con trỏ tương ứng được nhảy thẳng đến 6. Khi phủ màu 2 trên  $[4, 7)$ , con trỏ bắt đầu tại 4, nhận ra vùng đã tô, nhảy qua đến 6, chỉ tô ô 6, rồi dừng tại 7. Khi phủ màu 3 trên  $[1, 9)$ , con trỏ chỉ thật sự tô các ô còn trống và nhảy qua các cụm đã có màu.

Tinh túy của thuật toán là hợp nhất các ô rời rạc đã được tô liên kề, hoặc đánh dấu trước để lần sau chúng tự động được hợp nhất. Đây chính là tư tưởng phân tích khấu hao, gọi nhớ đến rừng tập rời nhau có nén đường đi và hợp nhất theo hạng. Tuy nhiên phiên bản trên chưa nén đường đi đầy đủ, cũng chưa hợp nhất theo hạng, nên còn có thể hoàn thiện.

## 2.3 Hoàn thiện bằng nén đường đi đầy đủ

Xem các đoạn đã tô kề nhau là một tập hợp. Khi tìm kiếm trong  $[l, r)$ , nếu gặp đoạn đã tô, ta muốn nhảy qua cả tập hợp đó trong thời gian khấu hao  $O(1)$ .

Bổ sung mảng  $p$ , ban đầu  $p[i] = i$ , giống con trỏ cha trong cấu trúc disjoint-set. Mảng  $next$  vẫn khởi tạo bằng  $next[i] = i + 1$ , và mảng  $color$  vẫn ghi màu của ô thứ  $i$ .

Ta định nghĩa:

- $i$  là nút đại diện cho ô rời rạc thứ  $i$ ;
- $p[i]$  là cha của nút  $i$ ;
- nút có  $p[i] = i$  là gốc của một cây.

Với nút không phải gốc, giá trị  $next[i]$  không còn ý nghĩa. Với gốc, ở phiên bản chưa xét hạng, ta có  $next[i] = i + 1$ .

Thuật toán chèn đoạn được viết lại:

1. Đặt  $i = l$ .
2. Nếu  $color[i]$  chưa có màu, tô màu cho nó.
3. Ghi lại vị trí hiện tại của con trở.
4. Nếu  $i > r - 1$ , chuyển sang bước cuối.
5. Nếu  $p[i] \neq i$ , đặt  $i = p[i]$ ; ngược lại đặt  $i = next[i]$ .
6. Quay lại bước 2.
7. Nén đường đi: với mọi vị trí  $x$  con trở đã đi qua, đặt  $p[x]$  trở về đại diện mới của tập hợp sau khi hợp nhất.

Ví dụ cùng các đoạn  $[2, 6)$ ,  $[4, 7)$ ,  $[1, 9)$ : sau lần phủ đầu, các nút 2 đến 5 được hợp nhất, gốc là 5. Lần phủ thứ hai bắt đầu tại 4, theo cha đến 5, rồi nhảy đến 6, tô 6, sau đó hợp nhất hai tập với gốc mới là 6. Lần phủ thứ ba tô 1, nhảy qua tập bắt đầu ở 2, rồi tô 7, 8 và hợp nhất toàn bộ các tập liên quan.

## 2.4 Thiết lập hạng

Hạng ở đây có thể hiểu là số nút trong một tập. Khi hợp nhất hai tập, ta gán cây nhỏ hơn vào cây lớn hơn; cùng với nén đường đi, độ phức tạp khâu hao của các thao tác tập rời nhau là gần  $O(1)$ . Trong thuật toán của bài này, chỉ cần sửa phần chọn gốc mới sau khi hợp nhất: chọn gốc của cây có hạng lớn nhất làm đại diện mới.

Khi dùng hạng, không còn nhất thiết  $next[i] = i + 1$  đối với gốc, nên cần duy trì riêng giá trị “vị trí tiếp theo sau cả tập”. Có nhiều nhất  $2N$  tập, số lần hợp nhất không vượt quá  $2N - 1$ . Vì vậy thời gian xử lý một hàng rời rạc là  $O(N)$ , và toàn bộ thuật toán đạt

$$O(N^2).$$

## 2.5 Tiểu kết

Từ bỏ cây đoạn, quay về thuật toán thô rồi cải tiến nó là điều không dễ. Quá trình hình thành thuật toán cũng rất chặt chẽ: trước hết tìm chỗ dư thừa để đạt cải thiện về lượng; sau khi thấy ưu điểm của cách làm, liên hệ nó với một cấu trúc dữ liệu tương tự, hấp thụ những ưu điểm như nén đường đi và hợp nhất theo hạng; cuối cùng đạt được thay đổi về chất trong độ phức tạp.

## 3 Kết luận

Trong truyền thuyết Hy Lạp, vua Phrygia thắt nút Gordius và tiên đoán rằng ai tháo được nút ấy sẽ thống trị châu Á. Alexander Đại đế không lần theo từng sợi dây, mà rút kiếm chém đứt nút. Có người nói đó là “gian lận”, nhưng có lẽ điều các vị thần cần tìm chính là sự quyết đoán ít lời, nhiều hành động như vậy.

Đối mặt với một thế giới đa dạng, ta thường rơi vào trạng thái “ý thức về đáp án quá mạnh”: nhanh chóng tưởng tượng ra một đáp án, vui vẻ đi theo con đường đó, nhưng không chú ý xem giả thiết ban đầu có đúng và đầy đủ hay không. Bị chính chướng ngại vô hình do mình tạo ra che mắt là điều đáng tiếc. Vì vậy ta không nên dừng bước vì nhận thức phiến diện hoặc vì sợ bỏ lợi ích trước mắt; đôi khi cần sự quả cảm để quay lại điểm xuất phát và định nghĩa lại vấn đề. Đó chính là sự thống nhất giữa tính tiến lên và tính quanh co của tư duy.

Cách biểu diễn vấn đề thường quan trọng hơn đáp án. Đáp án có thể chỉ là toán học hoặc thực nghiệm, còn việc đặt ra câu hỏi mới, khả năng mới, hoặc nhìn một vấn đề cũ từ góc độ mới mới đòi hỏi trí tưởng tượng sáng tạo. Trở về điểm xuất phát để định nghĩa lại vấn đề chính là một nguồn gốc của tiến bộ khoa học.

## Tài liệu tham khảo

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.
- Donald E. Knuth, *The Art of Computer Programming*.
- Allen B. Tucker et al., *The Computer Science and Engineering Handbook*.
- Wu Wenhui, Wang Jiande, *Phân tích và thiết kế chương trình cho thuật toán thực dụng*.
- Lời giải Baltic OI 2004 do Xin Tao cung cấp tại <http://get-me.to/oi>.

## Lời cảm ơn

Tác giả cảm ơn Xin Tao ở tỉnh Liêu Ninh vì sự giúp đỡ lớn, gồm một phần thuật toán và chứng minh; cảm ơn các thầy cô trong nhóm huấn luyện Olympic tin học thanh thiếu niên tỉnh Giang Tô; cảm ơn các thầy cô của Trường Ngoại ngữ Nam Kinh, gia đình, và tất cả bạn bè trong giới tin học.